Homework-6

1.Simulate the undo and redo option in word packages using a doubly Linked List.

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
struct node{
char text[100];
struct node *prev,*next;
};
struct node *current=NULL;
void save(char text[]){
struct node *n=malloc(sizeof(struct node));
strcpy(n->text,text);
n->prev=current;
n->next=NULL;
if(current){
struct node *t=current->next;
while(t){
struct node *temp=t;
t=t->next;
free(temp);
}
current->next=n;
}
current=n;
}
```

```
void undo(){
if(current&&current->prev)current=current->prev;
printf("Text: %s\n",current?current->text:"");
}
void redo(){
if(current&&current->next)current=current->next;
printf("Text: %s\n",current?current->text:"");
}
int main(){
save("Hello");
save("Hello World");
save("Hello World!");
undo();
undo();
redo();
return 0;
}
```

2.Given two sorted Lists L1 and L2 write a procedure to compute L1intersection L2 and L1Ul2 using basic list operations.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int data;
struct node *next;
};
struct node *create(int x){
```

```
struct node *n=malloc(sizeof(struct node));
n->data=x;
n->next=NULL;
return n;
}
struct node *append(struct node *head,int x){
struct node *n=create(x),*t=head;
if(!head)return n;
while(t->next)t=t->next;
t->next=n;
return head;
}
struct node *intersection(struct node *a,struct node *b){
struct node *r=NULL;
while(a&&b){
if(a->data==b->data){
r=append(r,a->data);
a=a->next;
b=b->next;
}else if(a->data<b->data)a=a->next;
else b=b->next;
}
return r;
}
struct node *unionList(struct node *a,struct node *b){
struct node *r=NULL;
```

```
while(a&&b){
if(a->data==b->data){
r=append(r,a->data);
a=a->next;
b=b->next;
}else if(a->data<b->data){
r=append(r,a->data);
a=a->next;
}else{
r=append(r,b->data);
b=b->next;
}
}
while(a){r=append(r,a->data);a=a->next;}
while(b){r=append(r,b->data);b=b->next;}
return r;
}
void display(struct node *head){
while(head){printf("%d ",head->data);head=head->next;}
printf("\n");
}
int main(){
struct node *a=NULL,*b=NULL,*r;
a=append(a,1);a=append(a,3);a=append(a,5);a=append(a,7);
b=append(b,3);b=append(b,5);b=append(b,6);b=append(b,8);
r=intersection(a,b);
```

```
printf("Intersection: ");display(r);
r=unionList(a,b);
printf("Union: ");display(r);
return 0;
}
```

OUTPUT:

Intersection: 3 5

Union: 1 3 5 6 7 8

3.Swap two elements by adjusting only the pointer (and not the data) using doubly Linked List.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int data;
struct node *prev,*next;
};
struct node *swap(struct node *head,int x,int y){
struct node *a=head,*b=head,*t;
while(a&&a->data!=x)a=a->next;
while(b&&b->data!=y)b=b->next;
if(!a||!b||a==b)return head;
if(a->next==b){
if(a->prev)a->prev->next=b;
else head=b;
if(b->next)b->next->prev=a;
a->next=b->next;
b->prev=a->prev;
```

```
b->next=a;
a->prev=b;
return head;
}
if(b->next==a)return swap(head,y,x);
t=a->prev;
a->prev=b->prev;
b->prev=t;
if(a->next)a->next->prev=b;
if(b->next)b->next->prev=a;
t=a->next;
a->next=b->next;
b->next=t;
if(a->prev)a->prev->next=a;
else head=a;
if(b->prev)b->prev->next=b;
else head=b;
return head;
}
void display(struct node *head){
while(head){printf("%d ",head->data);head=head->next;}
}
```

4.Write an array implantation of self adjusting lists .A self adjusting list is like a regular list ,except that all insertions are performed at the and when an element is accessed by a find it is moved to the front of the list, without changing the relative order of the other elements.

```
#include<stdio.h>
#define MAX 100
```

```
int a[MAX],n=0;
void insert(int x){
if(n<MAX)a[n++]=x;
}
int find(int x){
int i,j;
for(i=0;i<n;i++){
if(a[i]==x){
int temp=a[i];
for(j=i;j>0;j--)a[j]=a[j-1];
a[0]=temp;
return 1;
}
}
return 0;
}
void display(){
int i;
for(i=0;i<n;i++)printf("%d ",a[i]);
printf("\n");
}
int main(){
insert(10);
insert(20);
insert(30);
insert(40);
```

```
display();
find(30);
display();
return 0;
}
```

OUTPUT:

10 20 30 40

30 10 20 40

5.Write a c program to reverse the given doubly linked list.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int data;
struct node *prev,*next;
};
struct node *reverse(struct node *head){
struct node *cur=head,*temp;
while(cur){
temp=cur->prev;
cur->prev=cur->next;
cur->next=temp;
head=cur;
cur=cur->prev;
}
return head;
}
```

```
void display(struct node *head){
while(head){printf("%d ",head->data);head=head->next;}
}
```

6.Write a C program to delete a middle element in Doubly Linked List.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int data;
struct node *prev,*next;
};
struct node *deleteMiddle(struct node *head){
struct node *slow=head,*fast=head;
if(!head)return NULL;
while(fast&&fast->next){
slow=slow->next;
fast=fast->next->next;
}
if(slow->prev)slow->prev->next=slow->next;
else head=slow->next;
if(slow->next)slow->next->prev=slow->prev;
free(slow);
return head;
}
```

7.Remove a duplicate elements from a sorted Doubly linked list. And unsorted doubly linked list.

(i)Sorted DLL:

```
struct node *removeSorted(struct node *head){
```

```
struct node *cur=head,*temp;
while(cur&&cur->next){
if(cur->data==cur->next->data){
temp=cur->next;
cur->next=temp->next;
if(temp->next)temp->next->prev=cur;
free(temp);
}else cur=cur->next;
}
return head;
}
```

(ii) Unsorted DLL:

```
struct node *removeUnsorted(struct node *head){
struct node *cur=head,*p,*temp;
while(cur){
p=cur->next;
while(p){
if(p->data==cur->data){
temp=p;
if(p->prev)p->prev->next=p->next;
if(p->next)p->next->prev=p->prev;
p=p->next;
free(temp);
}else p=p->next;
}
cur=cur->next;
```

```
}
return head;
}
```

8.Calculate the frequency count of a given integer in a circular Doubly linked list.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int data;
struct node *prev,*next;
};
int frequency(struct node *head,int key){
struct node *temp;
int count=0;
if(head==NULL)return 0;
temp=head;
do{
if(temp->data==key)count++;
temp=temp->next;
}while(temp!=head);
return count;
}
```

9.Search for a particular elements in a binary tree.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int data;
```

```
struct node *left,*right;
};
int search(struct node *root,int key){
if(root==NULL)return 0;
if(root->data==key)return 1;
return search(root->left,key)||search(root->right,key);
}
```

10.Calculate the height or depth of a tree and print it.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int data;
struct node *left,*right;
};
int height(struct node *root){
int l,r;
if(root==NULL)return 0;
l=height(root->left);
r=height(root->right);
return 1+(l>r?l:r);
}
```

11.Given a binary tree. Convert this to a Doubly linked list. Keeping the same order(Inorder traversal)

```
#include<stdio.h>
#include<stdlib.h>
struct node{
```

```
int data;
struct node *left,*right;
};
struct node *head=NULL,*prev=NULL;
void convert(struct node *root){
if(root==NULL)return;
convert(root->left);
if(prev==NULL)head=root;
else{
root->left=prev;
prev->right=root;
}
prev=root;
convert(root->right);
}
void display(){
struct node *temp=head;
while(temp){
printf("%d ",temp->data);
temp=temp->right;
}
}
```

12.Given a binary tree the task is to connect the nodes that are at the same level using next right pointer.

```
#include<stdio.h>
#include<stdlib.h>
```

```
struct node{
int data;
struct node *left,*right,*nextRight;
};
void connect(struct node *root){
struct node *level,*cur,*next,*tail,*child;
if(!root)return;
root->nextRight=NULL;
level=root;
while(level){
cur=level;
next=NULL;
tail=NULL;
while(cur){
if(cur->left){
child=cur->left;
if(!next)next=child;
else tail->nextRight=child;
tail=child;
}
if(cur->right){
child=cur->right;
if(!next)next=child;
else tail->nextRight=child;
tail=child;
}
```

```
cur=cur->nextRight;
}
if(tail)tail->nextRight=NULL;
level=next;
}
}
```

13.Implement priority queue using binary search tree and perform operation insert.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int priority;
struct node *left,*right;
};
struct node *insert(struct node *root,int p){
if(root==NULL){
root=malloc(sizeof(struct node));
root->priority=p;
root->left=root->right=NULL;
}else if(p>root->priority)root->right=insert(root->right,p);
else if(p<root->priority)root->left=insert(root->left,p);
return root;
}
void display(struct node *root){
if(root){
display(root->right);
printf("%d ",root->priority);
```

```
display(root->left);
}
}
int main(){
struct node *root=NULL;
root=insert(root,30);
root=insert(root,50);
root=insert(root,10);
root=insert(root,40);
display(root);
return 0;
}
```

OUTPUT:

50 40 30 10

14.In a given binry search tree write routines to delete a node with Failed children ,one left child and two right child.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int data;
struct node *left,*right;
};
struct node *deleteNode(struct node *root,int key){
struct node *temp;
if(root==NULL)return NULL;
if(key<root->data)root->left=deleteNode(root->left,key);
```

```
else if(key>root->data)root->right=deleteNode(root->right,key);
else{
if(root->left==NULL){
temp=root->right;
free(root);
return temp;
}
if(root->right==NULL){
temp=root->left;
free(root);
return temp;
}
temp=root->right;
while(temp->left)temp=temp->left;
root->data=temp->data;
root->right=deleteNode(root->right,temp->data);
}
return root;
}
```

15.Convert the given postfix expression to an expression tree.

```
#include<stdio.h>
#include<stdlib.h>
#include<ctype.h>
struct node{
char data;
struct node *left,*right;
```

```
};
struct node *stack[100];
int top=-1;
struct node *create(char c){
struct node *n=malloc(sizeof(struct node));
n->data=c;
n->left=n->right=NULL;
return n;
}
struct node *build(char exp[]){
int i;
struct node *n;
top=-1;
for(i=0;exp[i]!='\0';i++){
if(isalnum((unsigned char)exp[i]))stack[++top]=create(exp[i]);
else{
if(top<1)return NULL;
n=create(exp[i]);
n->right=stack[top--];
n->left=stack[top--];
stack[++top]=n;
}
}
if(top!=0)return NULL;
return stack[top];
}
```

```
void inorder(struct node *root){
if(root){
if(root->left)printf("(");
inorder(root->left);
printf("%c",root->data);
inorder(root->right);
if(root->right)printf(")");
}
}
int main(){
struct node *root=build("ab+c*");
if(root)inorder(root);
return 0;
}
```

OUTPUT:

(a+b)*c

16.Construct binary search tree using (i) Inorder node representeation.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int data;
struct node *left,*right;
};
struct node *create(int x){
struct node *n=malloc(sizeof(struct node));
n->data=x;
```

```
n->left=n->right=NULL;
return n;
}
struct node *buildBalanced(int a[],int l,int r){
int mid;
struct node *root;
if(l>r)return NULL;
mid=(l+r)/2;
root=create(a[mid]);
root->left=buildBalanced(a,l,mid-1);
root->right=buildBalanced(a,mid+1,r);
return root;
}
void inorder(struct node *root){
if(root){
inorder(root->left);
printf("%d ",root->data);
inorder(root->right);
}
}
int main(){
int a[]={10,20,30,40,50};
struct node *root=buildBalanced(a,0,4);
inorder(root);
return 0;
}
```

(ii)Preorder and

```
#include<stdio.h>
#include<stdlib.h>
#include<limits.h>
struct node{
int data;
struct node *left,*right;
};
struct node *create(int x){
struct node *n=malloc(sizeof(struct node));
n->data=x;
n->left=n->right=NULL;
return n;
}
struct node *build(int a[],int *i,int min,int max,int n){
struct node *root;
int x;
if(*i>=n)return NULL;
x=a[*i];
if(x<=min||x>=max)return NULL;
root=create(x);
(*i)++;
root->left=build(a,i,min,x,n);
root->right=build(a,i,x,max,n);
return root;
}
```

```
void inorder(struct node *root){
if(root){
inorder(root->left);
printf("%d ",root->data);
inorder(root->right);
}
}
int main(){
int a[]={40,20,10,30,60,50,70};
int i=0,n=7;
struct node *root=build(a,&i,INT_MIN,INT_MAX,n);
inorder(root);
return 0;
}
```

OUTPUT:

10 20 30 40 50 60 70

(iii)postorder.

```
#include<stdio.h>
#include<stdlib.h>
#include<limits.h>
struct node{
int data;
struct node *left,*right;
};
struct node *create(int x){
struct node *n=malloc(sizeof(struct node));
```

```
n->data=x;
n->left=n->right=NULL;
return n;
}
struct node *build(int a[],int *i,int min,int max){
struct node *root;
int x;
if(*i<0)return NULL;
x=a[*i];
if(x<=min||x>=max)return NULL;
root=create(x);
(*i)--;
root->right=build(a,i,x,max);
root->left=build(a,i,min,x);
return root;
}
void inorder(struct node *root){
if(root){
inorder(root->left);
printf("%d ",root->data);
inorder(root->right);
}
}
int main(){
int a[]={10,30,20,50,70,60,40};
int i=6;
```

```
struct node *root=build(a,&i,INT_MIN,INT_MAX);
inorder(root);
return 0;
}
```

OUTPUT:

10 20 30 40 50 60 70